ELAW 프로젝트

DB 담당자 역할 정의서

데이터베이스 설계 · 관리 · AI 연동 전체 구조

담당자: 오 | 버전: v1.0 | 2026년 3월

항목	내용
문서 목적	ELAW 플랫폼 DB 담당자(오)의 역할·책임·작업 항목 전체를 우선순위와 함께 정의
작성 기준	ELAW PRD v1.0 (2026.03)
적용 범위	MySQL 스키마 설계, AI 모듈 연동, ETL 파이프라인, 테스트 데이터, 유지보수
협업 인터페이스	백엔드(김건) ↔ DB/AI(오) ↔ 프론트엔드(윤) — API·스키마 명세 공유

0. 우선순위 범례

이 문서의 모든 작업은 아래 4단계 우선순위로 분류된다. 개발 일정(4개월) 내 반드시 P0 → P1 순으로 완료하고, P2·P3는 여유 시 진행한다.

등급	의미	목표 완료 시점	비고
P0 필수	없으면 서비스 자체가 동작 불가 — 모든 기능의 기반	Sprint 1~2 (3월)	가장 먼저 착수
P1 중요	핵심 사용자 시나리오 구현에 직접 필요	Sprint 3~5 (4월~5월 초)	P0 완료 후 착수
P2 보통	사용자 경험 향상·품질 강화에 필요하나 없어도 데모 가능	Sprint 6~7 (5월)	일정 여유 시
P3 낮음	장기적 확장·최적화 항목, 졸업 과제 이후 적용 가능	Sprint 8 이후	시간 허용 시

1. DB 담당자 역할 개요

1.1 포지션 요약

DB/AI 담당자(오)는 ELAW 플랫폼에서 생성·수집·분석되는 모든 데이터의 구조를 설계하고, AI 분석 모듈의 입출력 파이프라인을 책임지는 역할이다. 백엔드 개발자(김건)가 API를 구현할 수 있도록 스키마와 인터페이스를 먼저 확정해야 하므로, 프로젝트 초반에 가장 먼저 산출물을 내야 하는 포지션이다.

1.2 책임 영역 한눈에 보기

영역	핵심 책임	협업 대상
DB 설계	전체 테이블 스키마 정의, 정규화, 인덱스 전략	백엔드(김건) — ORM 모델 기반
AI 모듈	Gemini API 연동 로직, 프롬프트 설계, 응답 파싱	팀장(김태) — 기능 요구사항 확인
ETL 파이프라인	외부 플랫폼 데이터 수집·정규화·적재 구조 설계	백엔드(김건) — 배치 스케줄러
데이터 분석	취약점 분석, 매칭 점수 계산, 성장 곡선 집계 로직	프론트엔드(윤) — 시각화 데이터
테스트 데이터	개발·시연용 더미 데이터 구축 및 관리	전원 — 개발·테스트 시 활용
유지보수	마이그레이션, 쿼리 최적화, 백업 정책	백엔드(김건) — 서버 인프라 협력

2. 전체 작업 목록 (우선순위 순)

아래는 DB 담당자가 수행해야 할 모든 작업을 우선순위 순서로 나열한 마스터 리스트이다. 각 항목의 상세 내용은 3~7장에서 설명한다.

우선순위	작업명	상세 내용	산출물
P0 필수	**DB** 스키마 초안 작성	전체 테이블·컬럼·타입·PK/FK 정의 (ERD 포함)	schema_v1.sql + ERD 이미지
P0 필수	핵심 테이블 **Migration** 생성	Django migration 파일 작성 및 백엔드 팀에 전달	migrations/ 파일
P0 필수	인증·사용자 테이블 확정	users, companies, sessions 테이블 — JWT 연동 기준	테이블 명세서
P0 필수	목표·커리큘럼 테이블 확정	user_goals, curricula 테이블 — Gemini 출력 JSON 저장 구조	테이블 명세서
P0 필수	풀이 이력 테이블 확정	solve_history, learning_stats 테이블 — 집계 기반 설계	테이블 명세서
P0 필수	**Gemini API** 연동 구조 설계	호출 방식, 프롬프트 템플릿, 응답 파싱 흐름 문서화	AI 연동 명세서
P1 중요	채용·매칭 테이블 확정	job_postings, matches, applications 테이블	테이블 명세서
P1 중요	포트폴리오 테이블 확정	portfolios, portfolio_sections 테이블	테이블 명세서
P1 중요	게시판 테이블 확정	posts, categories 테이블	테이블 명세서
P1 중요	**ETL** 파이프라인 구조 설계	백준·GitHub 수집 흐름, 정규화 규칙, 에러 처리 정의	ETL 명세서
P1 중요	인덱스 전략 수립	자주 쓰이는 쿼리 기준 인덱스 컬럼 선정	인덱스 설계서
P1 중요	**AI** 분석 로직 설계	취약점 분석, 매칭 점수, 커리큘럼 생성 알고리즘 설계	AI 분석 명세서
P1 중요	개발용 더미 데이터 구축	학생 10명·기업 3개·풀이 이력 500건 수준 시드 데이터	seed_data.sql
P2 보통	캐시 레이어 설계	Gemini 응답·통계 집계 결과 캐싱 전략 (Redis)	캐시 전략서
P2 보통	배치 스케줄 설계	일 1회 외부 수집, 주간 stats 집계 Cron 설계	배치 명세서
P2 보통	포트폴리오 **PDF** 저장 전략	파일 경로 규칙, 스토리지 용량 계획	파일 관리 명세
P2 보통	데이터 백업 정책	orion 서버 mysqldump 스케줄, 보관 기간 정의	백업 정책서
P3 낮음	쿼리 성능 최적화	EXPLAIN 분석 후 느린 쿼리 개선	최적화 보고서
P3 낮음	데이터 통계 대시보드용 뷰	관리자 통계 페이지용 MySQL View 정의	VIEW SQL

우선순위	작업명	상세 내용	산출물
P3 낮음	**AI** 모델 고도화 검토	규칙 기반 → 딥러닝 전환 가능성 조사	검토 보고서

3. P0 — 핵심 스키마 설계 (Sprint 1~2, 3월)

이 장의 모든 작업은 백엔드 팀이 API를 구현하기 전에 완료되어야 한다.

스키마가 확정되지 않으면 Django ORM 모델을 작성할 수 없어 전체 개발이 블로킹된다.

3.1 전체 테이블 목록 및 역할

테이블명	역할	주요 관계
users	학생·기업·관리자 계정 통합 관리	user_goals, solve_history, portfolios의 부모
companies	기업 회원 프로필 (관리자 승인 포함)	users(1:1), job_postings의 부모
user_goals	목표 조사 결과 저장 (분야·직무·기간)	users(N:1), curricula의 부모
curricula	Gemini 생성 커리큘럼 JSON 저장	user_goals(N:1)
solve_history	외부 플랫폼 문제 풀이 이력 원본	users(N:1)
learning_stats	언어·태그별 집계 지표 (배치 갱신)	users(1:1 per lang/tag)
portfolios	자동 생성 포트폴리오 메타 + 내용	users(N:1)
job_postings	기업 채용 공고	companies(N:1), matches의 부모
matches	AI 매칭 점수 및 지원 상태	users(N:1), job_postings(N:1)
posts	게시판 공지·이벤트·대회 정보	users(N:1, 관리자만 작성)
platform_links	외부 플랫폼 연동 계정 정보 (백준ID, GitHub)	users(N:1)
ai_logs	Gemini API 호출 이력 (디버깅·비용 추적)	users(N:1)

3.2 핵심 테이블 스키마 구조

아래는 각 테이블의 컬럼 구조를 정의한다. 실제 구현 시 Django migration으로 관리하며, 백엔드 담당자와 함께 ORM 모델로 변환한다.

① users — 통합 사용자 테이블

```
CREATE TABLE users (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  email         VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,            -- bcrypt
  name          VARCHAR(100) NOT NULL,
  role          ENUM('student','company','admin') NOT NULL DEFAULT
'student',
  phone         VARCHAR(20),
  is_active     TINYINT(1) NOT NULL DEFAULT 1,
```

```
  ai_consent    TINYINT(1) NOT NULL DEFAULT 0,    -- AI 이용 동의
  privacy_consent TINYINT(1) NOT NULL DEFAULT 0,
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);
```

② companies — 기업 프로필

```
CREATE TABLE companies (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT UNSIGNED NOT NULL UNIQUE,    -- FK → users.id
  name        VARCHAR(200) NOT NULL,
  industry    VARCHAR(100),
  description TEXT,
  logo_url    VARCHAR(500),
  is_approved TINYINT(1) NOT NULL DEFAULT 0,      -- 관리자 승인 여부
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

③ user_goals — 목표 조사

```
CREATE TABLE user_goals (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id       BIGINT UNSIGNED NOT NULL,
  goal_type     ENUM('study','job') NOT NULL,
  field         VARCHAR(100),                     -- 컴퓨터, 전기, 조선...
  job_role      VARCHAR(100),                     -- 백엔드, AI 개발자...
  start_date    DATE,
  end_date      DATE,
  mid_eval_date DATE,
  duration_weeks INT,
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

④ curricula — Gemini 생성 커리큘럼

```
CREATE TABLE curricula (
  id             BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  goal_id        BIGINT UNSIGNED NOT NULL,         -- FK → user_goals.id
  user_id        BIGINT UNSIGNED NOT NULL,
  content_json   JSON NOT NULL,                    -- Gemini 출력 그대로 저장
  version        TINYINT UNSIGNED NOT NULL DEFAULT 1,
  is_active      TINYINT(1) NOT NULL DEFAULT 1,
  created_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  FOREIGN KEY (goal_id) REFERENCES user_goals(id),
  FOREIGN KEY (user_id) REFERENCES users(id)
);

-- content_json 예시 구조:
-- { "weeks": [
--     { "week": 1, "theme": "배열·정렬",
--       "tasks": ["버블정렬 구현", "백준 #2750"], "hours": 8 },
--     ...
--   ]
-- }
```

⑤ solve_history — 풀이 이력 원본

```
CREATE TABLE solve_history (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT UNSIGNED NOT NULL,
  platform    ENUM('baekjoon','programmers','w3schools','custom') NOT NULL,
  problem_id  VARCHAR(50) NOT NULL,              -- 플랫폼 내 문제 번호
  problem_title VARCHAR(300),
  status      ENUM('solved','failed','partial') NOT NULL,
  language    VARCHAR(50),                       -- Python, Java, C++ ...
  algo_tags   JSON,                              -- ["정렬","DP","그래프"]
  difficulty  VARCHAR(20),                       -- Bronze5, Silver3 ...
  solved_at   DATETIME NOT NULL,
  source      ENUM('api','crawl','manual') DEFAULT 'api',
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_user_platform_problem (user_id, platform, problem_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

⑥ learning_stats — 집계 지표 (배치 갱신)

```
CREATE TABLE learning_stats (
  id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id         BIGINT UNSIGNED NOT NULL,
  stat_type       ENUM('language','algo_tag') NOT NULL,
  stat_key        VARCHAR(100) NOT NULL,           -- 'Python' 또는 '정렬'
  total_count     INT UNSIGNED NOT NULL DEFAULT 0,
  solved_count    INT UNSIGNED NOT NULL DEFAULT 0,
  correct_rate    FLOAT,                           -- solved / total * 100
  last_solved_at  DATETIME,
  updated_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  UNIQUE KEY uq_user_stat (user_id, stat_type, stat_key),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

```
);
```

⑦ portfolios — 자동 생성 포트폴리오

```
CREATE TABLE portfolios (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id       BIGINT UNSIGNED NOT NULL,
  title         VARCHAR(300),
  summary_text  TEXT,                           -- Gemini 생성 요약
  content_json  JSON,                           -- 섹션별 내용
  public_slug   VARCHAR(100) UNIQUE,            -- 공개 URL 식별자
  pdf_path      VARCHAR(500),                   -- 서버 내 PDF 경로
  language      ENUM('ko','en') DEFAULT 'ko',
  version       TINYINT UNSIGNED DEFAULT 1,
  is_public     TINYINT(1) DEFAULT 1,
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

⑧ job_postings — 채용 공고

```
CREATE TABLE job_postings (
  id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  company_id      BIGINT UNSIGNED NOT NULL,
  title           VARCHAR(300) NOT NULL,
  description     TEXT,
  required_skills JSON,                         --
["Python","Django","MySQL"]
  job_role        VARCHAR(100),
  deadline        DATE,
  is_active       TINYINT(1) DEFAULT 1,
  created_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE
);
```

⑨ matches — AI 매칭 결과 및 지원 현황

```
CREATE TABLE matches (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT UNSIGNED NOT NULL,
  posting_id  BIGINT UNSIGNED NOT NULL,
  match_score FLOAT,                            -- 0~100, AI 계산
  status      ENUM('recommended','applied',
               'viewed','scrapped','rejected') DEFAULT 'recommended',
  applied_at  DATETIME,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_user_posting (user_id, posting_id),
```

```
  FOREIGN KEY (user_id)    REFERENCES users(id),
  FOREIGN KEY (posting_id) REFERENCES job_postings(id)
);
```

⑩ platform_links — 외부 플랫폼 연동

```
CREATE TABLE platform_links (
  id           BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id      BIGINT UNSIGNED NOT NULL,
  platform     ENUM('baekjoon','github','programmers') NOT NULL,
  external_id  VARCHAR(200) NOT NULL,             -- 해당 플랫폼의
username/ID
  access_token VARCHAR(500),                      -- OAuth 토큰 (암호화 저장)
  last_synced  DATETIME,
  is_active    TINYINT(1) DEFAULT 1,
  UNIQUE KEY uq_user_platform (user_id, platform),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

⑪ posts — 게시판

```
CREATE TABLE posts (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  author_id   BIGINT UNSIGNED NOT NULL,           -- 관리자만 작성
  category    ENUM('notice','contest','event') NOT NULL,
  title       VARCHAR(300) NOT NULL,
  content     TEXT NOT NULL,
  is_pinned   TINYINT(1) DEFAULT 0,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  FOREIGN KEY (author_id) REFERENCES users(id)
);
```

⑫ ai_logs — AI 호출 이력

```
CREATE TABLE ai_logs (
  id           BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id      BIGINT UNSIGNED,
  feature      ENUM('curriculum','weakness','recommendation',
               'portfolio','matching','problem_gen') NOT NULL,
  prompt_tokens  INT,
  output_tokens  INT,
  latency_ms     INT,
  status         ENUM('success','error','timeout') DEFAULT 'success',
  error_message  VARCHAR(500),
  created_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
);
```

4. ERD 관계 요약

아래는 전체 테이블 간 관계를 텍스트로 정리한 것이다. 실제 ERD 다이어그램은 draw.io 또는 DBdiagram.io를 사용해 시각화하여 팀 Notion에 공유한다.

```
users (1) ──────── (N) user_goals
users (1) ──────── (N) solve_history
users (1) ──────── (N) learning_stats
users (1) ──────── (N) portfolios
users (1) ──────── (N) matches
users (1) ──────── (N) platform_links
users (1) ──────── (N) ai_logs
users (1) ──────── (1) companies

user_goals  (1) ── (N) curricula

companies   (1) ── (N) job_postings
job_postings(1) ── (N) matches

posts ── author_id → users (관리자)
```

※ 설계 원칙

· 모든 테이블은 BIGINT UNSIGNED AUTO_INCREMENT PK 사용

· 외래키 삭제 정책: 자식 데이터가 핵심인 경우 CASCADE, 로그성 테이블은 SET NULL

· ENUM 타입은 명세 확정 후 수정이 번거로우므로 초기에 충분히 검토

· JSON 컬럼은 MySQL 8.x 기준, 구조가 고정되면 정규화 테이블로 전환 검토

5. P1 — AI 모듈 & ETL 파이프라인 구조 (Sprint 3~5)

5.1 AI 모듈 전체 구조

AI 모듈은 백엔드 Django 서버 내부에 Python 모듈로 구현한다. Gemini API 호출은 모두 이 모듈을 통해서만 이루어지며, DB 담당자가 프롬프트 설계·응답 파싱·에러 처리 로직을 책임진다.

기능 모듈	입력 (DB 조회)	Gemini 역할	출력 (DB 저장)
커리큘럼 생성	user_goals + learning_stats	주차별 커리큘럼 JSON 생성	curricula.content_json
취약점 분석	solve_history (최근 90일)	취약 태그 Top-5 + 조언 텍스트	portfolios 또는 별도 캐시
문제 추천	learning_stats + 취약 태그	추천 문제 ID 리스트 생성	API 응답 (저장 불필요)
포트폴리오 생성	learning_stats + solve_history + GitHub	섹션별 서술 텍스트 생성	portfolios.content_json
매칭 점수 계산	learning_stats + job_postings.required_skills	TF-IDF 유사도 + 보정 점수	matches.match_score
문제 자동 생성	learning_stats (취약 유형)	문제 명세 + 예시 입출력 생성	API 응답 (저장 불필요)

5.2 Gemini API 호출 흐름

① 백엔드 API 엔드포인트 수신 (예: POST /api/goals)

② DB에서 사용자 학습 데이터 조회 (learning_stats, solve_history)

③ 프롬프트 빌더: 시스템 프롬프트 + 사용자 데이터 + Few-shot 예시 조합

④ Gemini 1.5 Pro API 호출 (백엔드 서버에서만, API 키 환경변수)

⑤ 응답 파싱: JSON 추출 → 유효성 검증 → 실패 시 최대 3회 재시도

⑥ 결과 DB 저장 (curricula 또는 portfolios)

⑦ ai_logs 테이블에 호출 이력 기록 (토큰 수, 지연 시간, 성공 여부)

⑧ 동일 입력 파라미터의 경우 캐시 적중 → DB 조회만 (API 미호출)

5.3 ETL 파이프라인 구조

외부 플랫폼 데이터 수집은 아래 3단계(Extract → Transform → Load)로 구성한다. Django management command로 구현하며, cron으로 일 1회 자동 실행한다.

단계	작업	세부 내용
Extract	백준 데이터 수집	solved.ac API 호출: /users/{handle}/problems — 풀이 문제 목록, 난이도, 태그
Extract	GitHub 데이터 수집	GitHub REST API: /users/{login}/repos, /repos/{owner}/{repo}/commits
Extract	수집 실패 처리	HTTP 에러·타임아웃 시 에러 로그 기록, 기존 데이터 보전, 다음 배치에서 재시도
Transform	플랫폼 데이터 정규화	문제 번호·태그·난이도를 내부 표준 형식으로 변환 (ENUM 매핑)
Transform	중복 제거	uq_user_platform_problem UNIQUE 제약으로 중복 INSERT 방지 (INSERT IGNORE)
Transform	언어 감지	파일 확장자 또는 플랫폼 제공 언어 정보를 내부 language 코드로 정규화
Load	solve_history 적재	신규 풀이 이력만 INSERT (기존 레코드 스킵)
Load	learning_stats 갱신	집계 쿼리로 correct_rate 재계산 후 UPSERT (INSERT ... ON DUPLICATE KEY UPDATE)
Load	platform_links 갱신	last_synced 타임스탬프 업데이트

6. P1 — 인덱스 전략

자주 실행되는 쿼리 패턴을 기준으로 인덱스를 선정한다. 과도한 인덱스는 INSERT 성능을 저하시키므로, 초기에는 아래 목록만 적용하고 이후 EXPLAIN 분석 결과에 따라 추가한다.

테이블	인덱스 컬럼	용도
users	email (UNIQUE)	로그인 시 이메일 검색
solve_history	user_id, solved_at	특정 사용자의 최근 풀이 조회
solve_history	user_id, platform, problem_id (UQ)	중복 방지 및 특정 문제 존재 확인
solve_history	algo_tags (FULLTEXT 또는 JSON 경로)	태그별 풀이 필터링
learning_stats	user_id, stat_type, stat_key (UQ)	UPSERT 성능 및 특정 통계 조회
matches	user_id, match_score DESC	사용자별 추천 공고 내림차순 조회
matches	posting_id, status	공고별 지원자 목록 조회
job_postings	company_id, is_active	기업별 활성 공고 조회
curricula	user_id, is_active	현재 활성 커리큘럼 조회
posts	category, created_at DESC	카테고리별 최신 게시글 조회

7. P1 — 개발·시연용 더미 데이터 구조

개발 초기부터 팀 전체가 실제 데이터처럼 보이는 테스트 데이터를 사용할 수 있도록, 아래 규모의 시드 데이터를 seed_data.sql로 준비한다.

테이블	데이터 수	내용
users	15건	학생 10명, 기업담당자 3명, 관리자 2명 (비밀번호: 'test1234' bcrypt 해시 통일)
companies	3건	승인된 기업 2개, 승인 대기 1개
user_goals	10건	학생 1인당 1개 목표 (공부 7명, 취업 3명)
curricula	10건	user_goals별 AI 생성 커리큘럼 JSON (수작업으로 구조 맞춰 작성)
solve_history	500건	학생 10명 × 50문제 — 백준 실버~골드 수준, 다양한 태그
learning_stats	30건	학생 10명 × 언어 3종 (Python, Java, C++) 통계
job_postings	6건	기업 3개 × 공고 2개씩 (백엔드, AI, 데이터 직무)
matches	20건	학생별 추천 공고 매칭 + 일부 지원 완료 상태
posts	5건	공지 2건, 대회 정보 2건, 이벤트 1건
platform_links	10건	학생 10명 백준 아이디 연동 (가상 핸들)

8. P2 — 캐시·배치·백업 설계

8.1 캐시 레이어 (Redis)

orion 서버에 Redis를 Docker 컨테이너로 추가 배포한다. 아래 데이터를 캐싱하여 Gemini API 비용과 DB 부하를 줄인다.

캐시 키 패턴	TTL	내용
curriculum:{user_id}:{goal_hash}	1시간	동일 목표 조사 입력에 대한 Gemini 응답
weakness:{user_id}	30분	취약점 분석 결과 텍스트
recommend:{user_id}	15분	오늘의 추천 문제 리스트
dashboard:{user_id}	5분	대시보드 집계 지표 (풀이 수, 완료율 등)
match_scores:{user_id}	1시간	채용 공고 매칭 점수 리스트

8.2 배치 스케줄

배치 작업	주기	Django Command	담당 역할
외부 플랫폼 데이터 수집	매일 새벽 3시	python manage.py sync_platforms	DB/AI 담당 설계, 백엔드 구현
learning_stats 재집계	매일 새벽 4시	python manage.py rebuild_stats	DB/AI 담당 설계 및 구현
만료 캐시 정리	매일 새벽 5시	Redis TTL 자동 만료	자동 처리
채용 공고 마감 처리	매일 자정	python manage.py expire_postings	DB/AI 담당 설계, 백엔드 구현

8.3 백업 정책

항목	정책
백업 도구	mysqldump (전체 덤프) — 4GB 미만 예상으로 충분
백업 주기	매일 1회 (새벽 2시 cron 실행)
보관 기간	최근 7일치 보관 → 이후 자동 삭제 (orion 서버 용량 고려)
저장 위치	orion 서버 내 /backup/mysql/ 디렉토리 (별도 파티션 권장)
복구 절차	mysql < backup_YYYYMMDD.sql 으로 복구, 백엔드 담당자와 함께 복구 테스트 1회 진행

9. 팀 내 협업 인터페이스

9.1 DB 담당자 → 백엔드 담당자 (김건)

전달 산출물 및 시점:

Sprint 1 종료 전 — schema_v1.sql (전체 CREATE TABLE 구문)

Sprint 1 종료 전 — ERD 이미지 (Notion 페이지에 업로드)

Sprint 2 착수 전 — Django migration 파일 초안 (DB 담당자 작성 → 백엔드 검토)

Sprint 3 착수 전 — ETL 파이프라인 명세서 (수집 흐름, 오류 처리 정책)

Sprint 4 착수 전 — AI 연동 명세서 (프롬프트 템플릿, 응답 JSON 스키마)

협의 필요 항목:

· ORM 모델 vs 직접 SQL 쿼리 사용 기준 합의

· 트랜잭션 처리 범위 (어느 레이어에서 커밋/롤백할지)

· Celery 도입 여부 (배치 처리 방식 확정)

9.2 DB 담당자 → 프론트엔드 담당자 (윤)

전달 정보:

· 대시보드 API 응답 JSON 구조 예시 (chart 라이브러리 바인딩 기준으로 설계)

· 레이더 차트 데이터: { labels: ["Python","Java",...], values: [85, 72, ...] }

· 꺾은선 그래프: { dates: [...], counts: [...] } (최근 30일 풀이 수)

· 포트폴리오 공개 슬러그 규칙: /portfolio/{public_slug} 형태로 라우팅

9.3 스키마 변경 프로세스

- 스키마 변경이 필요할 때는 반드시 팀 Notion의 'DB 변경 로그' 페이지에 먼저 기록
- 백엔드 담당자와 합의 후 migration 파일 생성 → PR 올려서 팀장 승인
- 운영 DB에 직접 ALTER TABLE 금지 — 반드시 migration으로만 변경
- 컬럼 삭제 시 2단계 진행: ① is_deprecated=True 컬럼 추가 후 배포 → ② 다음 스프린트에서 실제 삭제

10. 스프린트별 DB 담당자 체크리스트

Sprint 1 (3월 1~2주) — 설계

#	체크 항목	완료 기준
1	전체 테이블 목록 및 역할 확정	팀 전원 리뷰 완료
2	users·companies 테이블 스키마 작성	schema_v1.sql에 CREATE TABLE 작성 완료
3	user_goals·curricula 스키마 작성	Gemini JSON 구조 포함
4	solve_history·learning_stats 작성	UNIQUE 제약, 인덱스 포함
5	ERD 초안 작성 (draw.io)	Notion 페이지 업로드
6	백엔드 팀에 스키마 전달 및 리뷰	김건 확인 서명

Sprint 2 (3월 3~4주) — 핵심 기능 DB 지원

#	체크 항목	완료 기준
1	portfolios·job_postings·matches 스키마 확정	schema_v1.sql 최종본
2	Django migration 파일 생성 및 전달	python manage.py migrate 성공
3	더미 데이터 seed_data.sql 초안 작성 (users 위주)	로그인 테스트 가능 수준
4	Gemini API 호출 Python 함수 프로토타입	커리큘럼 JSON 1회 생성 성공

Sprint 3~4 (4월) — AI 엔진 & 연동

#	체크 항목	완료 기준
1	백준 ETL 수집 모듈 구현	실제 풀이 이력 DB 저장 성공
2	GitHub API 연동 모듈 구현	커밋 수·언어 통계 저장 성공
3	learning_stats 집계 배치 구현	correct_rate 값 정확 계산 확인
4	취약점 분석 AI 로직 구현	Top-5 태그 텍스트 생성 성공
5	포트폴리오 자동 생성 로직 구현	content_json DB 저장 성공
6	더미 데이터 solve_history 500건 입력	시각화 테스트 가능 수준

Sprint 5~6 (5월) — 취업 매칭 & 마무리

#	체크 항목	완료 기준
1	매칭 점수 계산 로직 구현	matches 테이블 score 값 저장 성공
2	인덱스 추가 및 EXPLAIN 분석	주요 쿼리 full scan 없음 확인

#	체크 항목	완료 기준
3	Redis 캐시 레이어 연동	동일 요청 2회째 캐시 히트 확인
4	백업 스크립트 작성 및 cron 등록	mysqldump 파일 생성 확인
5	배치 스케줄 cron 등록	새벽 배치 1회 이상 성공 실행
6	최종 더미 데이터 완성	시연 시나리오 전체 동작 가능

부록. 주요 규칙 및 참고 사항

A. 네이밍 컨벤션

대상	규칙	예시
테이블명	소문자 스네이크 케이스, 복수형	users, solve_history, job_postings
컬럼명	소문자 스네이크 케이스	created_at, user_id, algo_tags
PK	id (BIGINT UNSIGNED AUTO_INCREMENT)	모든 테이블 공통
FK	{참조테이블_단수형}_id	user_id, company_id, posting_id
Boolean	is_ 또는 has_ 접두사	is_active, is_approved, has_synced
타임스탬프	created_at, updated_at	DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
ENUM	소문자 영문	'student','solved','baekjoon'

B. 보안 유의 사항

- 비밀번호: bcrypt 해시 저장 (평문 절대 금지)
- OAuth 토큰 (platform_links.access_token): AES-256 암호화 후 저장
- ai_logs에 프롬프트 전문 저장 금지 — 토큰 수·상태·지연 시간만 기록
- 개인정보 컬럼(email, phone)은 SELECT 시 필요한 곳에서만 조회

C. 사용 도구

도구	용도
MySQL Workbench / DBeaver	로컬 DB 접속 및 스키마 확인
draw.io (diagrams.net)	ERD 다이어그램 작성 (무료, 웹 기반)
DBdiagram.io	DBML 문법으로 ERD 빠른 초안 작성
Postman	AI API 응답 포맷 검증
Notion	스키마 명세·변경 로그·체크리스트 관리